IIMT UNIVERSITY
MEERUT
Transforming Education System, Transforming Lives

UGC Approved
Section 2(f) & 12B

NAAC Accredited with A GRADE

CENTER FOR INNOVATION, INCUBATION & ENTREPRENEURSHIP
BUSINESS INCUBATOR
MEERUT

Powered by
MLH MAJOR LEAGUE HACKING x DEV

Presenting partner
DigitalOcean

AI

PROBLEM STATEMENTS

AI BELONGS TO EVERYONE

HACKTOBERFEST 2026
HACKDAY@MEERUT
X
ECELL IIMT UNIVERSITY



HACKTOBERFEST HACKDAY 2026

HACKDAY@MEERUT × E-CELL IIMTU

BUILD. TEST. CONTRIBUTE.

A focused challenge booklet for student innovators and open-source builders.

TRACK 1	TRACK 2
BEST USE OF GEMMA 4	BEST OPEN-SOURCE AI PROJECT

How to Use This Booklet

PS	TITLE	WHAT TEAMS BUILD	TRACK
PS 01	Study Companion from Real Notes	Learning support, evidence-grounded answers, quiz feedback	Track 1
PS 02	Notice-to-Action Accessibility Assistant	Extract facts, simplify, translate or provide accessible output	Track 1
PS 03	Community and Environmental Data Intelligence	Ask questions of a dataset; verify calculations and visualise results	Track 1
PS 04	Open-Source Contribution and Developer Workflow Agent	Repository evidence, contribution artifact, verification	Track 2
PS 05	Open-Model Evaluation and Workflow Reliability	Repeatable test set, baseline comparison, failure analysis	Track 2

Building Rules :

- Choose one of the published problem statements.

- If your idea does not align with the listed problem statements, explicitly identify it as an original innovation. Selection of such ideas is comparatively limited; choosing a listed problem statement is recommended.
- The event focuses on open-source AI and open-weight models.
- APIs may be used; however, the open-source/open-weight AI component should remain central where required by the official challenge.
- AI tools may be used during project development.
- Pull requests are not the basis for participation in Hacktoberfest 2026.

TRACK	PROBLEM STATEMENT	FOCUS
TRACK 1 · BEST USE OF GEMMA 4	PS 01	LEARNING SUPPORT

Adaptive Learning and Knowledge Support

Study Companion from Real Notes

PROBLEM BACKGROUND

Students often revise from lectures, notes, and short PDFs that may not be easy to understand. A generic chatbot may answer from its own knowledge even when the answer is absent from the study material. This makes it difficult for learners to identify what they know, what they misunderstood, and which statements are supported by their notes.

PROBLEM STATEMENT

Build an AI-powered study companion that turns user-provided learning material into an interactive tutoring session. Students should be able to supply a short set of notes, ask questions, receive explanations grounded in that material, and test their understanding through practice questions and targeted feedback. For the Hack Day, focus on one academic topic and a few pages of material rather than a full learning-management system.

EXPECTED SOLUTION

Develop a web, desktop, or command-line tool that supports the following learning flow:



Core Requirements

- Allow students to upload or paste notes on one selected topic.
- Answer questions using the supplied material and show identifiable supporting evidence.
- Generate relevant practice questions and evaluate student responses.
- Explain incorrect answers and provide hints or guidance.
- Clearly indicate when the notes do not contain enough information to answer a question.
- Use Gemma 4 meaningfully to interpret the material and support learning.

Demonstration Checklist

- Load sample notes on one topic.
- Ask one question that is supported by the notes and show the evidence.
- Answer a generated quiz question incorrectly and display the follow-up explanation.
- Ask one unsupported question and show how the system communicates its limitation.

Possible Directions

- Explain supplied notes, documents, diagrams, or study material.
- Generate practice activities, feedback, or learning plans.
- Answer questions using identifiable evidence from the supplied material.
- Adapt explanations or difficulty to the learner's progress.

CORE CHALLENGE Use Gemma 4 to support learning; show how unsupported questions and output quality are handled.

TRACK	PROBLEM STATEMENT	FOCUS
TRACK 1 · BEST USE OF GEMMA 4	PS 02	ACCESSIBILITY

Accessible and Multilingual AI Assistance

Notice-to-Action Accessibility Assistant

PROBLEM BACKGROUND

College notices, community posters, government announcements, and application instructions can be dense, image-based, or written in a language the reader finds difficult. People may miss key details such as deadlines, required documents, or next steps. Accessibility requires more than translation: important facts must be extracted, simplified, and explained without inventing missing information.

PROBLEM STATEMENT

Create an assistant that converts a real-world notice or form instruction into an understandable action checklist. Users may upload a photo or screenshot, or paste text; select a language or simpler reading level; and receive an accessible explanation of what the notice requires. Preserve critical facts while making the next steps easier to follow. Use publicly shareable or synthetic notices for the demonstration.

EXPECTED SOLUTION

Build a minimal application that transforms a notice into clear, actionable information and supports at least one follow-up question.



Core Requirements

- Accept at least one image-based notice or text notice and extract its key contents.
- Show structured fields such as purpose, eligibility, deadline, required documents, contact, and action; label fields that are not stated.
- Provide a concise plain-language summary and an ordered checklist of actions.
- Offer output in at least two selected languages, or in one language plus a demonstrably accessible alternative such as read-aloud using suitable tooling.
- Handle a clarification question and require confirmation before treating uncertain extracted details as facts.
- Use Gemma 4 centrally to interpret information and guide the user.

Demonstration Checklist

- Display a sample scholarship, campus-event, or public-service notice.
- Show the simplified checklist in two modes, such as two languages or a language and accessible alternative.
- Ask about a requirement that is not stated and demonstrate that the assistant reports the information is absent.

Possible Directions

- Explain documents, images, or interfaces in accessible language.
- Support multilingual interaction using suitable supporting tools.

- Offer voice, text, or visual interfaces for the selected audience.
- Guide users through tasks with clarification and confirmation.

CORE CHALLENGE Use Gemma 4 to interpret information and guide users; demonstrate accessibility and limitations.

TRACK	PROBLEM STATEMENT	FOCUS
TRACK 1 · BEST USE OF GEMMA 4	PS 03	DATA INTELLIGENCE

Community and Environmental Data Intelligence

PROBLEM BACKGROUND

Useful local data on electricity consumption, water usage, waste collection, air quality, mobility, or campus resources often sits in spreadsheets that are difficult for non-specialists to interpret. Charts without explanations can hide missing observations or incorrect comparisons. Community members need to ask questions in everyday language and receive traceable numerical answers with clear visual explanations.

PROBLEM STATEMENT

Create a data-exploration assistant for one defined community or environmental issue. The user loads a small CSV or structured dataset—for example, monthly electricity usage by campus building—and asks practical questions such as “Which building used the most electricity last month?” The tool should interpret the question, calculate the answer from the supplied data, and present a chart, numerical result, and relevant caveats. Clearly distinguish sample or synthetic datasets from official public records.

EXPECTED SOLUTION

Build an interactive data question-answering prototype in which model-generated interpretation is grounded in executable calculations.



Core Requirements

- Load one documented CSV or structured dataset with several records and clearly named columns.
- Answer at least three useful question types, such as highest/lowest, trend, and comparison between periods or groups.
- Calculate totals, averages, and changes using code or trusted data operations; do not rely solely on a model-generated number.
- Display at least one suitable chart or table alongside a plain-language explanation, and cite the underlying rows or fields.
- Warn when requested information is absent, a denominator is missing, or the data cannot justify the conclusion.
- Use Gemma 4 to connect user questions with evidence and useful analysis.

Demonstration Checklist

- Load a small campus-resource dataset.
- Ask for a ranking and a comparison between periods.
- Ask for an unavailable statistic and show how the tool handles missing information.
- Display the visual result and the actual calculation.

Possible Directions

- Combine documents, images, and structured data around one chosen issue.
- Let users explore data through natural-language questions.

- Explain patterns, missing information, and possible actions.
- Use tools for calculations, visualisations, or scenario comparisons.

CORE CHALLENGE Use Gemma 4 to connect questions with evidence; state assumptions and verify calculations in code.

TRACK	PROBLEM STATEMENT	FOCUS
TRACK 2 · BEST OPEN-SOURCE AI PROJECT	PS 04	DEVELOPER WORKFLOW

Open-Source Contribution and Developer Workflow Agent

PROBLEM BACKGROUND

New contributors often struggle to understand an unfamiliar repository, connect an issue to relevant files, and determine which tests to run. Maintainers also face repetitive questions and low-quality proposed fixes that are difficult to review. An evidence-backed assistant can reduce this friction without claiming that automated code suggestions have already been verified.

PROBLEM STATEMENT

Build an AI agent that takes a real public open-source repository—or a clearly labelled small demo repository—and one issue or development task, then guides the contributor towards a reviewable next step. It should inspect the project structure and README, identify likely source or documentation locations, propose a bounded fix or contribution plan, and report which verification steps were actually completed. The goal is a useful contribution artifact, not a generic conversation about GitHub.

EXPECTED SOLUTION

Create a CLI or small interface that turns a repository URL or local path and a specific issue into an evidence-grounded contribution plan with verifiable output.



Core Requirements

- Use an openly licensed repository and display its languages, entry points, contributor instructions, and relevant directories.
- Accept a selected issue or task and cite the repository files or documentation used to understand it.
- Produce a step-by-step plan and at least one reviewable artifact, such as a documentation diff, small patch, test case, or issue-reproduction checklist.
- Run an available check when practical, such as a test, linter, or example command. Distinguish passed, failed, and not-run checks.
- Explain the model's role and any limitations in repository access or file coverage.
- Use open-source or open-weight AI meaningfully in the workflow.

Demonstration Checklist

- Point the tool at a small repository and a documentation or bug issue.
- Show cited file locations and the generated artifact or diff.
- Display at least one real verification result.

Possible Directions

- Understand repository structure, documentation, or issue requirements.
- Help contributors identify suitable issues and prepare contributions.
- Reproduce problems, propose changes, or run relevant checks.

- Generate reviewable documentation, reports, or development artifacts.

CORE CHALLENGE Ground recommendations in repository evidence and verify changes with appropriate checks.

TRACK	PROBLEM STATEMENT	FOCUS
TRACK 2 · BEST OPEN-SOURCE AI PROJECT	PS 05	MODEL EVALUATION

Open-Model Evaluation and Workflow Reliability

PROBLEM BACKGROUND

An AI assistant can appear impressive on carefully selected examples and still fail on ordinary or ambiguous tasks. Developers need small, repeatable tests that expose mistakes, show whether instructions were followed, and make claimed improvements inspectable. Without a defined test set and visible scoring, model comparisons remain anecdotal.

PROBLEM STATEMENT

Create a compact evaluation kit for one open-source or open-weight AI model, or an agent that uses one. Choose a narrow workflow—for example, answering questions from supplied context, classifying support issues, or selecting the correct tool. Build a fixed test set, run a baseline, change one meaningful part of the workflow (such as the prompt, retrieval, context, or tool policy), and compare the recorded outputs. A result showing no improvement is acceptable when supported by evidence.

EXPECTED SOLUTION

Develop an executable evaluation script or notebook with a reusable test set and an understandable results report.



Core Requirements

- Define at least eight diverse test cases, including ordinary, ambiguous, and deliberate failure-inducing inputs.
- Write an explicit scoring rubric—for example, factual support, correct tool choice, or completed steps—with expected evidence.
- Run an open model or workflow and save responses, settings, timestamps or run identifiers, and scores.
- Compare two model/workflow variants, or a baseline against a modified configuration, using the same test set.
- Produce a simple table or chart, document two failure examples, and interpret limitations honestly.
- Make the evaluation reproducible and keep open-source or open-weight AI central to the project.

Demonstration Checklist

- Run a reproducible subset of tests live.
- Show baseline versus modified-workflow scores.
- Inspect one concrete failure record and explain what it reveals.

Possible Directions

- Evaluate answer quality, evidence use, robustness, or fairness.
- Test tool selection, task completion, and failure handling.
- Compare model or workflow alternatives on a shared task set.
- Record traces and explain where the system succeeds or fails.

CORE CHALLENGE Provide reusable tests, clear results, and evidence for claimed improvements.

Final Submission & Publication Checklist

Use this checklist to prepare a clear, reproducible submission and a student-friendly public repository.

- Choose one problem statement and define a focused, demonstrable use case.
- Build the minimum required workflow before optional extensions.
- Use the required AI model or open-source/open-weight AI approach for the selected track.
- Include sample inputs and clearly label synthetic data where applicable.
- Demonstrate a normal case and the relevant limitation, failure, or unsupported case.
- Show evidence for answers, calculations, repository changes, or evaluation scores.
- Publish a public GitHub repository with an open-source licence.
- Write a README explaining the project, how to run or try it, and the model and key dependencies used.
- Record limitations honestly; distinguish checks that passed, failed, and were not run.
- Insert and test the official event page, registration, and submission links before publishing the booklet.
- Check the final PDF on desktop and mobile for readable text, working links, and no clipped content.